tjenko

IMPLEMENTATION PLAN

Building the connection layer

v2

Reconciling the live Cursor-Battle build with the updated agentic event creation, three-mode profiles, full attendee lifecycle, and host CRUD & finale flows.

Prepared for **Lumenar** · Tjenko

Copenhagen · June 2026

CONTENTS

What's inside

0 What changed and what this covers

1 The decision that gates everything: state leaves the device

2 Target architecture

3 Data model

4 Flow 1 — Agentic event creation (host)

5 Flow 2 — Attendee profile and event lifecycle

6 Flow 3 — Host management, mapping CRUD & finale

7 The three profile modes

8 The agent pipeline

9 GDPR & consent

10 Realtime and offline

11 Integrations

12 Build phases

13 What I need to lock

0 What changed and what this covers

This plan synthesises four inputs: the **company brief** (the vision — an agent that owns the networking outcome), the **Cursor Battle experience doc** (the locked mechanics — one-scan pairing, coverage + serendipity, inclusive gamification, GDPR), the **technical handoff** (the app already running — Next.js App Router, a useReducer store, concrete scoring, keyword-overlap matching), and your **new requirements** (agentic event creation, three-mode profiles, the full attendee lifecycle, host CRUD + badge confirmation + a gamified finale).

The approach: keep everything the running app already got right, layer the new flows on top, and flag the one architectural change that is now unavoidable.

1 The decision that gates everything: state has to leave the device

The running app persists its entire `EventState` to `localStorage` under `"tjenko-event-state"`. That is fine for a single-device demo, but it breaks the product's core moment. When A scans B's QR, **both phones must instantly surface the same icebreaker** — and two phones cannot share a localStorage. The scan handshake, the live leaderboard, the host dashboard, and the big screen are all multi-device, realtime surfaces.

> THE GATING CHANGE
>
> Production needs a **server-authoritative, realtime store**. Recommendation: **Supabase** (Postgres + Realtime + Auth + Storage + Row-Level Security) — it covers, in one stack, the realtime channel the scan handshake needs, auth for persistent profiles, object storage for generated banners, and RLS to enforce GDPR visibility at the row level. Firebase is the viable alternative.

Crucially, this is **not a rewrite**. Keep the existing useReducer shape as the client-side cache; the change is that the reducer now hydrates from Postgres instead of localStorage, writes mutations through to the server, and subscribes to a realtime channel per active event. The component tree barely moves.

2 Target architecture

- **Frontend:** Next.js App Router, Tailwind, Framer Motion, HTML5 QR — unchanged. (The prototype is a single-file React artifact for iteration speed; it maps 1:1 onto the routes below.)
- **State:** server-authoritative (Supabase), the existing reducer as client cache, one realtime subscription per live event.
- **Agent service:** server endpoints calling Claude for event-content generation, profile enrichment, deck matching, icebreaker generation, and post-event badge assignment — server-side so keys and prompts never ship to the client.
- **Banner generation:** an image model for the hero banner; the prototype generates a themed SVG banner from an AI-returned palette as a stand-in.

- **Integrations:** Luma import, Google/Outlook Calendar, social publishing (LinkedIn / X / Instagram / Facebook), transactional email for invites and follow-ups.

Route map (App Router)

```
/                       landing
/auth                   sign in / sign up
/onboard                quick profile + consent
/me                     profile (Formal / Informal / Semiformal)
/me/passport            cross-event stats, connections, badges
/host                   host home + controls
/host/create            agentic event creation (4 steps)
/host/event/[id]        console: CRUD on mapping, live dashboard
/host/event/[id]/finale badge confirm + gamified conclusion
/e/[id]                 attendee event page: register → confirm → calendar
/e/[id]/deck            the live loop: deck → scan → icebreaker → confirm
/screen/[id]            big-screen view (leaderboard / live feed / finale)
```

3 Data model (extends the handoff doc, doesn't replace it)

Keep `Attendee`, `DeckCard`, `Connection`, `Score`, `EventConfig` as they are. Add and extend:

- **User** — profileMode (formal | informal | semiformal), three contact sets (§7), and cross-event lifetime fields: eventsAttended, totalConnections, lifetimeScore, badges[]. This is the persistent-passport spine; capture it now even though recommendations are phase 2.
- **Event** — bannerUrl, seedPrompt, generatedDescription, agenda[], status (draft | published | live | concluded), source (manual | luma), socialTargets[], calendarPushed, hostId.
- **Registration** — joins userId ↔ eventId with status (invited | registered | confirmed | checked-in | attended). Powers invite emails, attendance confirmation, and the "match against who's actually present" rule.
- **Mapping** — the host-editable assignment graph: the DeckCard edges the agent produced, exposed for full CRUD before going live (§6).
- **Badge** — a definition plus Award { badgeId, userId, status: suggested | confirmed }, so the agent auto-suggests and the host confirms (§6).

Carry the handoff doc's scoring and tiers verbatim so the new work stays consistent with the running app:

```
base connection +10 · icebreaker +5 · full deck +25 · beyond-deck +8 · streak +3/
consecutive · bridge ≤+5
tiers — Spectator 0 · Tjenkor 10 · Connector 25 · Super-Connector 50 · Room MVP 80
```

4 Flow 1 — Agentic event creation (host)

The host turns one line into a published event. Four steps, agent-driven:

1. **Prompt.** The host describes the event in a single line. One tap fires the agent.
2. **Banner.** The agent returns a title, tagline, palette, and vibe, and an image model renders a hero banner. Regenerate or nudge the palette. (Prototype: a themed SVG banner from the returned palette.)
3. **Refine.** The same generation returns a full description and a structured, timed agenda — both editable inline. The agent drafts, the host tunes.
4. **Publish.** Connect Luma (or stay native), push to the host's calendar, toggle the socials to announce on, and roll invite emails to the whole guest list. Publishing flips status to published.

The agent endpoint takes the seed prompt and returns one JSON object — `{ title, tagline, description, agenda[], palette, vibe }` — that populates every field at once. Generation is a single round trip; refinement is local editing on top.

5 Flow 2 — Attendee profile and event lifecycle

Profile setup (post sign-up)

A short onboarding picks the profile mode (§7), then the user fleshes out the relevant fields. The "what I want tonight" goal chips (collaborator / hiring / job-hunting / co-founder / fun) stay the single biggest lever on match quality and are mandatory.

Event lifecycle

From an event page the attendee can **register → confirm attendance → add to calendar**. Confirmation moves their Registration status and opts them into the night's matching. Invites arrive by email at publish; the event lands on their calendar; the agent maps them against fellow confirmed attendees and primes the deck.

The live loop (multi-device)

Each attendee carries **one personal QR** (their identity). They flip a face-down card to reveal a target + "why you two," find them, **scan the target's QR**, and the icebreaker surfaces on **both** phones (one of: Tech Debate · Building Swap · Hot Take · Rapid Fire · Show & Tell). Both tap "we met" to bank points; the card flips to done; the next unlocks. "Worth keeping?" feeds the quality signal.

The passport

The profile shows lifetime stats: **events attended on Tjenko**, **total connections across all events**, lifetime score, tier, and badges — every connection scored by the rules in §3. This is the cross-event record that compounds.

6 Flow 3 — Host management, mapping CRUD, and the gamified finale

Agentic mapping + follow-up

Everyone who opts in or confirms is mapped against fellow attendees by the agent, and the pairings are reinforced by email so the networking continues beyond the room.

Full CRUD on the dynamics

The host console exposes the assignment graph for editing: rearrange who's in whose deck, add or remove a pairing, swap a target, re-run matching for a subset. The agent proposes; the host can override.

Post-event badges — agent assigns, host confirms

When the event ends, the agent computes the superlatives from the night's data — Fastest Five · Most Adventurous · Best Icebreaker · People's Champion · Deck Finisher · Early Bird — and presents them as suggested awards. The host reviews and **confirms** (or reassigns) each before they go out.

The conclusion

The event closes on the big screen with the leaderboard, the confirmed superlatives, and the headline number ("Tonight this room made 180+ connections"). Each attendee gets a recap card; the host gets the outcome report that sells the next event.

7 The three profile modes

A single toggle changes what the profile surfaces and what's shared at an event. Visibility is consented per mode — others only see what the mode exposes and the user agreed to.

- **Formal** — professional only. Work/business email, LinkedIn, role, company, professional goals. For conferences, recruiting, founder-investor rooms. Matching leans on role, stack, stated goal.
- **Informal** — personal. Active handles (Instagram, X, WhatsApp, Facebook), hobbies, personality. For community socials and mixers where people connect as people. Matching leans on shared interests and vibe.
- **Semiformal** — a curated blend: the professional spine plus a selected slice of the personal. The default for builder events like the Cursor Battle, where people are there to work and to vibe.

Each mode carries its own GDPR posture: Informal exposes more sensitive personal contact, so its consent copy and retention defaults are tighter, and a minimal-profile option is always available.

8 The agent pipeline

One service, five jobs, all server-side:

1. **Event content** — seed prompt → title, tagline, description, agenda, palette (Flow 1).
2. **Enrichment** — GitHub first (public API; languages and recent repos are gold for builder rooms), LinkedIn via user-provided URL with confidence-matching before anything is attached. Show people what was found; let them correct it.
3. **Matching** — the handoff doc's keyword-overlap scoring, wrapped in coverage (everyone gives ~5 / receives ~5, no orphans), relevance weighting ("hiring ↔ job-hunting" is a strong edge), the serendipity dial (~65), live re-balance at check-in, and symmetry bias so mutual finds are likely.
4. **Icebreakers** — pre-generated and cached for assigned cards at T-1; generated on the fly (with a generic fallback) for unplanned beyond-deck pairs.
5. **Badges** — compute superlatives from the night's data; output suggestions for host confirmation.

9 GDPR & consent (non-negotiable, EU)

Carry the experience doc's bar and extend it for the new surfaces. Explicit, un-pre-ticked opt-in at join. Plain-language privacy notice at onboarding. Consented visibility with a minimal-profile option. "A friend is attending" alerts **double-consented and off by default**. Data minimisation with a configurable deletion window for raw social data. Confidence-matched enrichment only. The new additions — calendar push, social publishing, invite/follow-up email — each need their own opt-in, and the three profile modes get mode-specific consent copy (Informal is the strictest).

10 Realtime and offline

The scan handshake and leaderboard run over a realtime channel per event. Antler-grade Wi-Fi with 50–100 phones means the web app must **queue scans locally and sync when the connection returns** — degrade, don't break. If data is thin or scanning flakes, the serendipity dial goes to 100 (balanced random) and the "meet five people" game still works. Venue QR goes on printed signage and table tents, not just the screen.

11 Integrations

- **Luma** — import the guest list (names + emails) at create time; auto-enrich from there. Surface the privacy notice on the Luma page too.
- **Calendar** — Google/Outlook push when an attendee confirms, and for the host at publish.
- **Socials** — publish the announcement to the host's connected LinkedIn / X / Instagram / Facebook at the publish step.
- **Email** — invites at publish (rolled to the whole audience), pairing nudges before/during, recap and outcome report after.

12 Build phases

- **Phase 0 — Prototype (now).** Every new flow demonstrated end to end in a single-file React app: agentic event creation (live generation + themed banner), three-mode profile, attendee register → confirm → live scan loop with real scoring, host console with mapping CRUD, and the badge-confirm + gamified finale.
- **Phase 1 — Backend + realtime + auth.** Move state off localStorage to Supabase; wire the reducer to Postgres + realtime; real auth and persistent profiles.
- **Phase 2 — Agent service.** Stand up the five endpoints; replace prototype mocks with real enrichment, matching, content, icebreaker, and badge generation.
- **Phase 3 — Integrations.** Luma, calendar, socials, email.
- **Phase 4 — Passport + recommendations.** Cross-event accrual, "people like you are going to X," consented "your colleague is attending."

13 What I need to lock

DECISIONS TO UNBLOCK THE BUILD

1. **Aesthetic direction.** The running app is brutalist beige; the prototype is the dark "night-event" system. Pick one to converge on — the prototype keeps dark for now.
2. **Backend choice** — Supabase (recommended) or Firebase, so Phase 1 can start.
3. **Profile-mode default** — Semiformal as the builder-event default, confirm.
4. **Real image-gen** for banners, or themed/template banners for v1?
5. **Which socials** are in scope for publishing at launch (LinkedIn + X first?).
6. **Enrichment posture** — GitHub-API-primary with user-provided LinkedIn URL, confirmed.